



Arab Security Conference

2025
7th - 8th SEPTEMBER



Mohammad Khreasha

Cybersecurity Directory @ ScopeSky Communications

Hunting Threats with YARA



Agenda

- ⚡ **Introduction**
- ⚡ **YARA Installation**
- ⚡ **Using YARA**
- ⚡ **Anatomy of YARA Rules**
- ⚡ **Writing YARA Rules**
- ⚡ **Real World Scenarios**
- ⚡ **Q&A**



Arab Security Conference

2025
7th - 8th SEPTEMBER

Introduction



What is YARA?

⚡ **YARA is a pattern-matching engine designed to help analysts create rules that identify malicious files, processes, memory artifacts, & more.**

⚡ **Think of it as a customizable signature language for threat detection.**

⚡ **Pattern matching:**

strings (ASCII, UCS-2)

regular expressions

binary patterns (hex strings)





Why use YARA?

- ⚡ **Detect malware (based on signatures, code fragments, or obfuscation patterns).**
- ⚡ **Scan files & memory for threats.**
- ⚡ **Threat hunting in logs, binaries, or system artifacts.**
- ⚡ **Reverse engineering support (identify malware families).**
- ⚡ **Automation in SOCs & EDRs.**



What YARA Rules Good For?

- ⚡ **Malware classification** : Detect families, variants, or specific samples of malware by matching unique patterns.
- ⚡ **Threat hunting** : Search for IOCs across files, endpoints, or memory dumps in incident response.
- ⚡ **Memory scanning** : Identify malicious code or injected payloads in live memory.
- ⚡ **File identification** : Classify files based on signatures (e.g., PE, ELF, APK, malicious docs).
- ⚡ **IOC detection** : Match hashes, strings, patterns from threat intelligence feeds.
- ⚡ **Forensic triage** : Narrow down suspicious files in large datasets during DFIR.
- ⚡ **Hunting APT & toolkits** : Detect C2 beacons, custom packers, RATs, webshells, etc.
- ⚡ **Custom signatures** : Unlike AV, YARA allows analysts to define their own rules quickly.



Arab Security Conference

2025
7th - 8th SEPTEMBER

YARA Installation



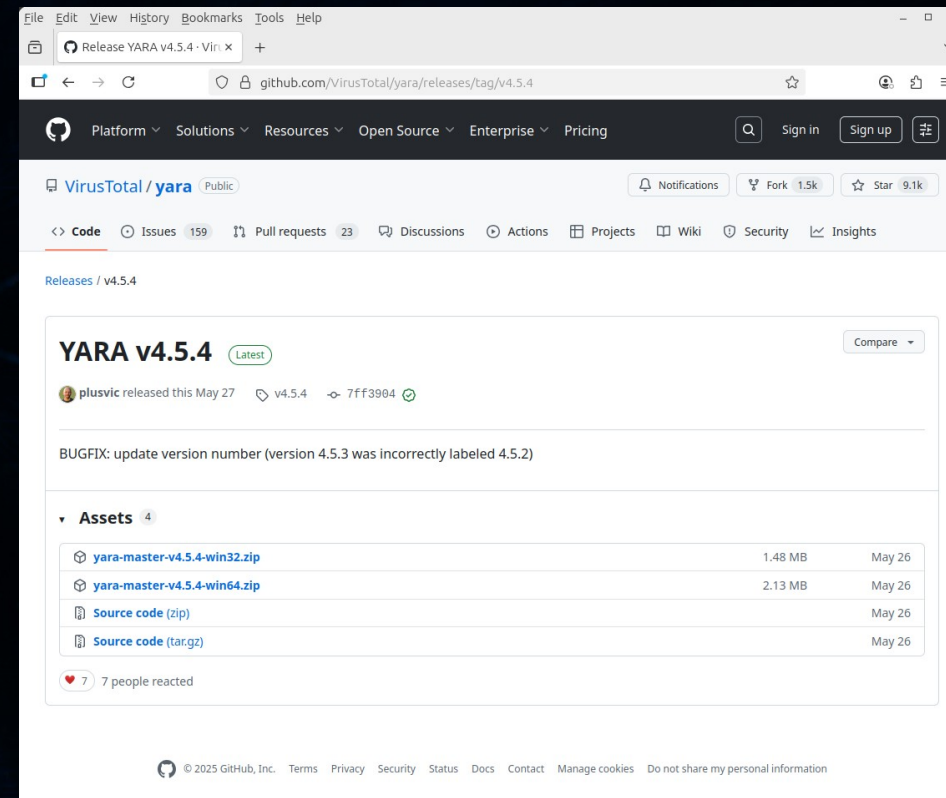
Installing YARA

⚡ The best way to install YARA on Windows OS is by obtaining the compiled binaries from the following github repository:

<https://github.com/VirusTotal/yara/releases/tag/v4.5.4>

⚡ Just extract the ZIP to a folder like:

C:\Tools\YARA\





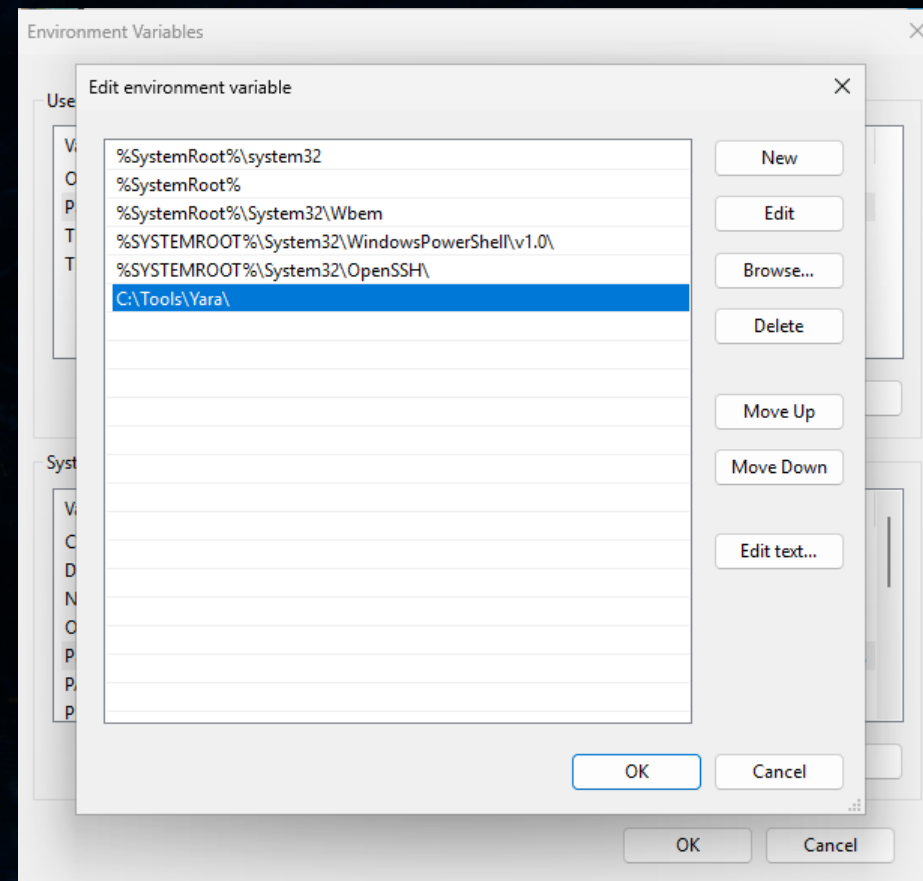
Continue...

⚡ Add **C:\Tools\YARA** to your **PATH** so you can run **yara.exe** from any terminal.

⚡ **Note :**

yara.exe → runs rules against files/directories.

yarac.exe → compiles **.yar** rules into optimized **.yarc** files.





Arab Security Conference

2025
7th - 8th SEPTEMBER

Using YARA



YARA Help

⚡ To get YARA help, you can use --help option, after yara32.exe or yara64.exe executables :

```
Command Prompt
C:\Users\technawi>yara64 --help
YARA 4.5.4, the pattern matching swiss army knife.
Usage: yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR | PID

Mandatory arguments to long options are mandatory for short options too.

      --atom-quality-table=FILE      path to a file with the atom quality table
-C,  --compiled-rules              load compiled rules
-c,  --count                       print only number of matches
-E,  --strict-escape               warn on unknown escape sequences
-d,  --define=VAR=VALUE            define external variable
-q,  --disable-console-logs        disable printing console log messages
      --fail-on-warnings            fail on warnings
-f,  --fast-scan                   fast matching mode
-h,  --help                        show this help and exit
-i,  --identifier=IDENTIFIER       print only rules named IDENTIFIER
      --max-process-memory-chunk=NUMBER set maximum chunk size while reading process memory (default=1073741824)
-l,  --max-rules=NUMBER            abort scanning after matching a NUMBER of rules
      --max-strings-per-rule=NUMBER set maximum number of strings per rule (default=10000)
-x,  --module-data=MODULE=FILE     pass FILE's content as extra data to MODULE
-n,  --negate                      print only not satisfied rules (negate)
```



Common YARA Options

Option	Usage	Example	Description
-r	Recursive scanning	yara.exe -r rules.yar C:\Samples\	Scan all files in subdirectories.
-s	Show matching strings	yara.exe -s rules.yar bad.exe	Prints offsets + strings that matched.
-c	Count matches only	yara.exe -c rules.yar C:\Samples\	Prints only the number of matching rules.
-d var=value	Define variables	yara.exe -d filename=bad.exe rule.yar bad.exe	Pass variables into rules (used in conditions).
-w	Ignore warnings	yara.exe -w rules.yar bad.exe	Continues even if rules have warnings.
-l	Print matching rule identifier only	yara.exe -l rules.yar bad.exe	Only outputs rule name, not file path.
-p N	Threads for scanning	yara.exe -p 4 rules.yar C:\Samples\	Uses 4 threads for faster scanning.
-f	Fast scan mode	yara.exe -f rules.yar bigfile.bin	Skips some matches, faster results.



Usage Example

⚡ Scan recursively & show matches

```
Command Prompt
C:\Users\technawi>yara64.exe -r -s c:\Tools\Yara\detect_malware.yar c:\users\technawi\desktop
C:\Users\technawi>
```



Arab Security Conference

2025
7th - 8th SEPTEMBER

Anatomy of YARA Rules



Anatomy of YARA rules

- ⚡ Understanding the anatomy of YARA rules is fundamental to creating reliable detection logic that minimizes both false positives and false negatives.
- ⚡ A YARA rule generally consists of five key components:

Imports (optional)

Rule Name (and Tags)

Metadata

Strings

Condition (including advanced conditions)

```
import "pe"
rule rule_name {
  meta:
    description = "detect a ransomware"
    author = "Mohammad Khreesha"
    date = "6/2/2025"
  strings:
    $s1 = "hacker.xyz" wide
    $s2 = "No system is safe" ascii
    $s3 = "vssadmin Delete Shadows /all /quiet" ascii
    $s4 = {6A 8B EC 6A FF 68 ?? ?? 42 00 64 A1 00 00 00 50 8? EC}
  condition:
    pe.imports("Shell32.dll", "ShellExecuteW") and 3 of them
}
```



Anatomy of YARA rules - Imports

⚡ YARA rules can be extended with modules, which provide additional functionality.

Module	Description	Key Features / Functions
pe	Analyzes Windows Portable Executable (PE) files.	Access entry point, section names, imports/exports, version info, number of sections, etc.
elf	Analyzes Linux ELF (Executable and Linkable Format) files.	Inspect entry point, section headers, symbol tables, etc.
cuckoo	Parses reports generated by Cuckoo Sandbox .	Extracts API calls, signatures, and behavioral indicators from sandbox analysis.
hash	Computes hashes of files or data segments.	MD5, SHA1, SHA256 functions; can be used for exact sample detection.
math	Provides mathematical functions.	abs(), sqrt(), sin(), cos(), log(), etc., for advanced condition building.
time	Extracts file timestamps (on-disk).	Access creation time, modification time, etc. Useful for detecting timestamping anomalies.
dotnet	Analyzes .NET executables.	Checks for .NET metadata, types, method names, and assembly information.
magic	Identifies file types based on “magic” numbers (like the Unix file command).	Detects PDFs, images, Office docs, archives, etc.
console	Provides print() functionality (for debugging rules).	Helps debug conditions and see intermediate values during testing.



Anatomy of YARA rules – Rule Name

- ⚡ Every YARA rule begins with a unique rule name. You can also assign tags (optional), separated by spaces, to help classify or group rules.
- ⚡ Tags make it easier to filter rules when scanning large datasets.
- ⚡ Example :

```
rule Suspicious_PE_File : windows malware trojan
{
    ...
    ...
    ...
    ...
}
```



Rule Naming Conventions

Aspect	Convention	Example(s)
Allowed Characters	Start with a letter or underscore . Only letters, numbers, underscores. No spaces/hyphens.	✓ Malware_Emotet_HTTP ✗ 123-malware
Case Style	Use a consistent style: CamelCase or snake_case.	Suspicious_PS_Script, malware_emotet_http
Descriptive Naming	Clearly describe purpose/focus of rule.	✓ Trojan_Emotet_Dropper ✗ Rule1
Naming Pattern	[Category]_[Platform/Family]_[Technique/Indicator].	Malware_Windows_Trickbot_Persistence
Categories (Prefixes)	Use Malware, APT, Suspicious, Exploit, Tool, Trojan, etc.	APT29_Windows_Dropper
Versioning	Add suffix for version/variant.	Emotet_HTTP_v2, APT28_Dropper_VariantA
Tags (Optional)	Tags classify but aren't part of rule name.	rule Malware_Emotet_HTTP : windows trojan banking



Anatomy of YARA rules – Meta

⚡ **Meta (Metadata) contains informational fields that don't affect rule logic but add important context.**

⚡ **Common metadata fields:**

author – Who wrote the rule.

description – Purpose of the rule.

date – Creation or modification date.

reference – External sources, MITRE ATT&CK techniques, or reports.

```
rule ransomware_detection {  
  meta:  
    description = "detect a ransomware attacks"  
    author = "Mohammad Khreesha"  
    date = "6/8/2025"  
    reference = "ATT&CK T1027.002"  
}
```



Meta Naming Conventions

Field	Convention	Example(s)
author	Your name or team name.	"John Doe", "ThreatHunt_Team"
description	Short summary of detection purpose.	"Detects Emotet dropper payload"
reference	URLs, CVEs, or threat reports.	"https://malware-traffic-analysis.net/", "CVE-2023-1234"
date	Use YYYY-MM-DD format.	"2025-09-06"
hash	Add sample hashes used in rule creation/testing.	"sha256:abc123..."
version	Increment with changes to rule.	"1.2"



Anatomy of YARA rules – Strings

⚡ Strings define the patterns YARA should look for. These may be:

⚡ Plain text strings

```
$cmd = "cmd.exe"
```

⚡ Hexadecimal byte patterns

```
$mz = { 4D 5A } // "MZ" header of PE files
```

⚡ Regular expressions

```
$url = /http:\\[a-z0-9\\.\\]+/ nocase
```



Strings Naming Conventions

Aspect	Convention	Example(s)
Prefix	All string identifiers start with \$.	\$str_cmd
Allowed Characters	Letters, numbers, underscores only.	\$url1, \$hex_payload
Case Sensitivity	Identifiers are case-sensitive; use consistent lowercase/snake_case.	\$dll_import
Descriptive Naming	Use names that explain what the string is. Avoid \$a, \$b.	\$powershell_exec, \$mutex_name
Group by Type	Use prefixes: str_ (text), re_ (regex), hex_ (hex).	\$str_api, \$re_ip, \$hex_shellcode
Private Strings	Mark helper-only strings with private.	\$helper_mutex private
Sequential Naming	For multiple related strings, use base name + index.	\$api_call_1, \$api_call_2



Anatomy of YARA rules – Modifiers

- ⚡ **Modifiers are options you can apply to strings in the strings section to change how they are matched.**
- ⚡ **They let you handle different encodings, case sensitivity, obfuscation, or control whether the string is considered in the rule logic.**

Modifier	Description	Example
nocase	Makes string matching case-insensitive.	"cmd.exe" nocase → matches CMD.EXE, Cmd.Exe.
ascii	Matches string in ASCII encoding (default).	"powershell" ascii
wide	Matches UTF-16LE (wide/Unicode) encoded version of the string.	"powershell" wide → matches p\0o\0w\0...
xor	Matches string XOR-encoded with a single-byte key (0x00–0xFF).	"malware" xor
fullword	Matches string only as a whole word, not as part of another word.	"net" fullword → matches net, not internet.
private	Excludes string from any of them / all of them, usable only in conditions.	"debug" private
base64	Matches Base64-encoded form of the string.	"malware" base64
base64wide	Matches Base64-encoded UTF-16 (wide) form of the string.	"malware" base64wide



Anatomy of YARA rules – Conditions

⚡ The condition determines when the rule triggers. This is where you combine string matches, file attributes, and logical operators.

⚡ Common operators :

any of them → At least one string matched.

all of them → All strings matched.

n of them → A specific number of strings matched.

Boolean operators: and, or, not.



YARA Rule Example

```
import "pe"

rule Malware_Windows_Emotet_HTTP : windows trojan banking
{
    meta:
        author      = "Technawi SOC Team"
        description = "Detects Emotet dropper making HTTP requests"
        reference    = "https://malware-traffic-analysis.net/"
        date         = "2025-08-06"
        version      = "1.0"

    strings:
        $str_powershell = "powershell.exe" wide nocase
        $str_cmd         = "cmd.exe" nocase
        $re_url          = /http:\/\/[a-z0-9\-\.\.]+\.[a-z0-9]+/ nocase
        $hex_shellcode   = { 90 90 90 ?? E8 ?? ?? ?? ?? }

    condition:
        uint16(0) == 0x5A4D and
        ( $str_powershell or $str_cmd ) and
        #re_url > 2 and
        $hex_shellcode
}
```



Arab Security Conference

2025
7th - 8th SEPTEMBER

Writing YARA Rules



Working with Files & their Contents

- ⚡ **YARA is widely used to identify, classify, and detect files based on their content, structure, and metadata.**
- ⚡ **YARA can analyze executable files, documents, scripts, and logs by matching strings, regex patterns, file headers, or binary sequences.**
- ⚡ **Principles for File Analysis in YARA :**
 - ⚡ **Magic bytes / File headers – Identify files using their signature bytes (PE headers, PDF %PDF-, JPG FFD8FF).**
 - ⚡ **File content strings – Detect keywords, suspicious patterns, or known malicious content.**
 - ⚡ **Regular expressions – Useful for matching dynamic patterns, URLs, or encoded content.**
 - ⚡ **PE/Document modules – Use pe for executables or magic/hash modules for content detection.**
 - ⚡ **File size or entropy checks – Reduce false positives by focusing on realistic file sizes or high-entropy binaries.**



Portable Executable (PE) Files

⚡ The Portable Executable (PE) format is used by Windows for executables (.exe), object code (.obj), and dynamic-link libraries (.dll).

⚡ Because most Windows malware is distributed in PE format, YARA is often applied to detect malicious PEs based on headers, sections, imports, and strings.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	DOS header
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	80	00	00	00	DOS stub
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	PE signature, PE file header
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	
00000080	50	45	00	00	4C	01	03	00	8D	FA	81	4D	00	00	00	00	
00000090	00	00	00	00	E0	00	02	01	0B	01	08	00	00	0A	00	00	PE standard fields
000000A0	00	08	00	00	00	00	00	00	9E	28	00	00	00	20	00	00	
000000B0	00	40	00	00	00	00	40	00	00	20	00	00	00	02	00	00	
000000C0	04	00	00	00	00	00	00	00	04	00	00	00	00	00	00	00	PE NT fields
000000D0	00	80	00	00	00	02	00	00	01	82	00	00	03	00	40	85	
000000E0	00	00	10	00	00	10	00	00	00	00	10	00	00	10	00	00	
000000F0	00	00	00	00	10	00	00	00	00	00	00	00	00	00	00	00	Data directories
00000100	4C	28	00	00	4F	00	00	00	00	40	00	00	A8	05	00	00	
00000110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000120	00	60	00	00	0C	00	00	00	A4	27	00	00	1C	00	00	00	.text section header
00000130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000150	00	00	00	00	00	00	00	00	00	20	00	00	08	00	00	00	.rsrc section header
00000160	00	00	00	00	00	00	00	00	00	08	20	00	00	48	00	00	
00000170	00	00	00	00	00	00	00	00	00	2E	74	65	78	74	00	00	
00000180	A4	08	00	00	00	20	00	00	00	0A	00	00	00	02	00	00	.reloc section header
00000190	00	00	00	00	00	00	00	00	00	00	00	00	00	20	00	00	
000001A0	2E	72	73	72	63	00	00	00	A8	05	00	00	00	40	00	00	
000001B0	00	06	00	00	00	0C	00	00	00	00	00	00	00	00	00	00	.text section
000001C0	00	00	00	00	40	00	00	40	2E	72	65	6C	6F	63	00	00	
000001D0	0C	00	00	00	00	60	00	00	00	02	00	00	00	12	00	00	
000001E0	00	00	00	00	00	00	00	00	00	00	00	00	00	40	00	00	
000001F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000200	80	28	00	00	00	00	00	00	48	00	00	00	02	00	05	00	
00000210	E4	20	00	00	C0	06	00	00	09	00	00	00	01	00	00	06	
00000220	00	00	00	00	00	00	00	00	50	20	00	00	80	00	00	00	
00000230	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	



Example #1 – Executable Files (PE)

⚡ Detecting suspicious Windows executables based on PE header and strings.

⚡ This example :

⚡ Checks for PE header (MZ).

⚡ Detects common Windows API functions often used in malware.

```
import "pe"

rule Suspicious_PE_File : executable malware windows
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects suspicious Windows PE files with common malware strings"
        date = "2025-08-06"

    strings:
        $s1 = "CreateProcessA" nocase
        $s2 = "VirtualAlloc" nocase
        $s3 = "kernel32.dll" nocase
        $hex_pattern = { 4D 5A ?? ?? ?? 00 00 00 }

    condition:
        uint16(0) == 0x5A4D and // PE header magic bytes
        any of ($s*) or
        $hex_pattern
}
```



Example #2 – Packed or Obfuscated PE

⚡ Malware often uses packers like UPX or modifies section names.

⚡ This example :

⚡ Detects uncommon section names.

⚡ Flags high entropy sections (often packed/encrypted).

```
import "pe"

rule Packed_PE_Suspicious : pe malware packed
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects potentially packed or obfuscated PE malware"
        date = "2025-08-06"

    condition:
        pe.is_pe and
        (
            for any section in pe.sections : (
                section.name not in (".text", ".rdata", ".data", ".rsrc", ".reloc")
                or section.entropy > 7.5
            )
        )
}
```




Example #3 – Office Word Files

⚡ Detects malicious macros or embedded URLs in Word files.

⚡ This example :

⚡ Detects OLE-based DOC files with macros or embedded URLs.

⚡ Can be extended to XLS/XLSX or DOCX (XML) detection.

```
rule Malicious_Word_File : doc malware
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects Word documents with macro malware or embedded URLs"
        date = "2025-08-06"

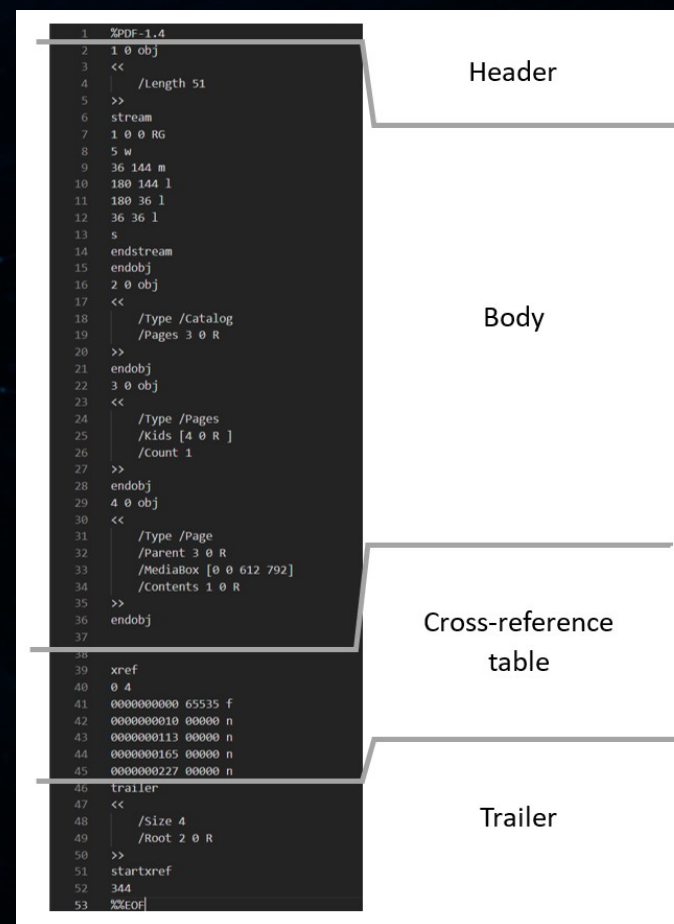
    strings:
        $macro = "AutoOpen" nocase
        $url = /http[s]?:\//[a-z0-9\.\-]+/ nocase
        $doc_header = { D0 CF 11 E0 A1 B1 1A E1 } // OLE header

    condition:
        $doc_header and ( $macro or $url )
}
```



PDF File Structure

- ⚡ PDF files support a wide variety of data types that can be present.
- ⚡ Threat actors fully control the content of the files they send to lure victims and they use the different capabilities of the PDF format for their attacks.
- ⚡ PDF files natively support JavaScript, so attackers can create files that will execute scripts once a file has been opened at this stage to download additional payload or steal information.
- ⚡ There are many PDF readers that are being used, some are multiplatform, others based on web browsers but like any other software they have bugs and vulnerabilities.





Example #4 – PDF Files

⚡ **Detects PDFs containing embedded JavaScript or objects used for exploits.**

```
rule Malicious_PDF : pdf malware
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects PDF files with JavaScript or suspicious objects"
        date = "2025-08-06"

    strings:
        $pdf_header = "%PDF-"
        $js = "/JavaScript" nocase
        $openaction = "/OpenAction" nocase
        $obj_stream = "/ObjStm" nocase

    condition:
        $pdf_header at 0 and any of ($js, $openaction, $obj_stream)
}
```



Detecting Networking Activities

- ⚡ **YARA was originally designed for file scanning, but it is also powerful for network artifacts when applied to:**
 - ⚡ **PCAP files (captured packets)**
 - ⚡ **Network payload dumps**
 - ⚡ **Logs containing HTTP/DNS requests**
 - ⚡ **Memory dumps of network-active processes**



Example #1 – User-Agents

⚡ Detect automated reconnaissance tools in web traffic logs or PCAPs.

```
rule Recon_HTTP_UserAgents : network recon
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects suspicious recon tools via HTTP User-Agents"
        date = "2025-08-06"

    strings:
        $curl = "User-Agent: curl" nocase
        $wget = "User-Agent: Wget" nocase
        $nikto = "Nikto/" nocase
        $sqlmap = "sqlmap" nocase

    condition:
        any of ($curl, $wget, $nikto, $sqlmap)
}
```



Example #2 – Data Exfiltration

⚡ Detect large Base64 blobs or archives being uploaded/downloaded.

```
rule Data_Exfiltration_HTTP : network exfil
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects exfiltration attempts via Base64 or archive transfer over HTTP"
        date = "2025-08-06"

    strings:
        $base64 = /[A-Za-z0-9+\\/{200,}={0,2}/
        $zip = "PK\\x03\\x04" // ZIP header
        $rar = "Rar!" // RAR header

    condition:
        any of ($base64, $zip, $rar)
}
```




Example #3 – DNS Tunneling/C2 Communication

⚡ Detect DNS-based tunneling or Cobalt Strike/Empire C2 traffic.

```
rule Suspicious_DNS_Tunneling : network dns c2
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects suspicious long DNS queries (tunneling, C2 domains)"
        date = "2025-08-06"

    strings:
        $dns_tunnel = /[a-z0-9]{50,}\.com/
        $cobaltstrike = "microsoftupdate" nocase
        $empire = "api.php?id=" nocase

    condition:
        any of ($dns_tunnel, $cobaltstrike, $empire)
}
```



Detecting OS Manipulations

- ⚡ OS manipulation refers to malware or attackers altering OS components to gain persistence, escalate privileges, disable security features, or hide their presence.
- ⚡ YARA can detect suspicious binaries, scripts, or configuration changes indicative of OS manipulation.
- ⚡ Principles for File Analysis in YARA :
 - ⚡ File/PE Analysis – Detect suspicious binaries replacing system files.
 - ⚡ Strings & Patterns – Detect references to registry keys, services, or system paths.
 - ⚡ Encoded/Obfuscated Scripts – Detect PowerShell, Bash, or Python scripts manipulating the OS.
 - ⚡ Behavioral Patterns – Detect actions like privilege escalation or driver installation.



Example #1 – Windows Registry Manipulation

- ⚡ Detects malware targeting registry keys for persistence or disabling defenses.
- ⚡ Combines critical OS registry keys with execution commands.

```
rule Suspicious_Registry_Modification : os_manipulation windows
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects Windows malware attempting to modify critical registry keys for persistence or security bypass"
        date = "2025-08-06"
        reference = "https://attack.mitre.org/techniques/T1547/"

    strings:
        $reg_run = "HKLM\\Software\\Microsoft\\Windows\\CurrentVersion\\Run" nocase
        $reg_policy = "DisableAntiSpyware" nocase
        $reg_firewall = "EnableFirewall" nocase
        $powershell = "powershell.exe" nocase
        $cmd = "cmd.exe" nocase

    condition:
        any of ($reg_run, $reg_policy, $reg_firewall) and any of ($powershell, $cmd)
}
```



Example #2 – Windows Service Manipulation

⚡ Detects service creation/modification targeting critical Windows paths.

```
rule Suspicious_Service_Modification : os_manipulation windows
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects malware attempting to create or modify services for persistence"
        date = "2025-08-06"

    strings:
        $sc_create = "sc create" nocase
        $sc_config = "sc config" nocase
        $service_path = "C:\\Windows\\System32\\" nocase
        $svc_name = "SysUpdater" nocase

    condition:
        any of ($sc_create, $sc_config) and $service_path and $svc_name
}
```




Example #3 – Linux Cron/Script Manipulation

⚡ **Detects Linux malware modifying startup scripts, cron jobs, or systemd services.**

```
rule Suspicious_Linux_OS_Modification : os_manipulation linux
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects Linux scripts that modify cron jobs or system config files"
        date = "2025-08-06"

    strings:
        $cron_add = "crontab -e" nocase
        $rc_local = "/etc/rc.local" nocase
        $systemd = "systemctl enable" nocase
        $bash_script = ".sh" nocase

    condition:
        any of ($cron_add, $rc_local, $systemd) and $bash_script
}
```



Spying Detection

⚡ **Spyware, keyloggers, and RATs are malware types that steal sensitive information or provide remote control over a system.**

Malware Type	Description	Common Indicators
Spyware	Monitors user activity silently, often stealing credentials or browsing habits.	Key API calls (GetAsyncKeyState, ReadFile), suspicious URLs, encrypted config files.
Keylogger	Records keystrokes, clipboard data, or captures screenshots.	Calls to GetAsyncKeyState, SetWindowsHookEx, writes log files.
RAT (Remote Access Trojan)	Provides full remote access/control over a victim system.	Hardcoded C2 IPs/domains, backdoor functionality, network activity, unusual registry modifications.



Example #1 - Keylogger

⚡ Detects keyloggers by API usage and log file references.

```
import "pe"

rule Suspicious_Keylogger_Windows : keylogger windows spyware
{
    meta:
        author      = "Technawi SOC Team"
        description = "Detects Windows keyloggers using common API calls and log file names"
        date         = "2025-08-06"
        reference    = "https://attack.mitre.org/techniques/T1056/"

    strings:
        $s1 = "GetAsyncKeyState" nocase
        $s2 = "SetWindowsHookEx" nocase
        $s3 = "KeyboardLogger" nocase
        $s4 = "C:\\Users\\Public\\Logs" nocase

    condition:
        any of ($s*) and filesize < 5MB
}
```



Example #2 - RAT

- ⚡ Detects RATs using backdoor API calls, persistence registry entries, or C2 domains.
- ⚡ `uint16(0) == 0x5A4D` ensures the file is a PE binary.

```
import "pe"

rule Suspicious_RAT : rat spyware remote
{
    meta:
        author      = "Technawi SOC Team"
        description = "Detects generic RATs using hardcoded C2 domains or suspicious PE
imports"
        date        = "2025-08-06"

    strings:
        $url_c2 = /[a-z0-9\-\]+\.\com/i
        $$_backdoor = "CreateRemoteThread" nocase
        $$_persist = "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run" nocase

    condition:
        uint16(0) == 0x5A4D and // PE header
        any of ($$_backdoor, $$_persist) or
        $url_c2
}
```




Example #3 - Spyware

⚡ Detects spyware that encodes sensitive data before exfiltration.

⚡ \$b64 regex detects long Base64 sequences.

```
rule Spyware_Encoded_Config : spyware obfuscated
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects spyware with Base64 encoded configuration strings"
        date = "2025-08-06"

    strings:
        $b64 = /[A-Za-z0-9+\\\/]{50,}={0,2}/
        $s_exfil = "SendData" nocase
        $s_capture = "CaptureScreen" nocase

    condition:
        $b64 and any of ($s_exfil, $s_capture) and filesize < 6MB
}
```



Ransomware Detection

- ⚡ **Ransomware is malicious software designed to encrypt files or disrupt systems to demand a ransom.**
- ⚡ **Ransomware often exhibits characteristic behaviors, including:**
 - ⚡ **File system activity:** Creating or modifying many files rapidly.
 - ⚡ **File extensions:** Appending unusual extensions (e.g., .locky, .crypt).
 - ⚡ **Strings:** Hardcoded ransom notes, encryption routines, or C2 addresses.
 - ⚡ **API usage:** Calls to cryptographic functions (CryptEncrypt, AES, RSA).
 - ⚡ **PE characteristics:** Packed sections, abnormal imports, or unusual entry points.



Example #1

⚡ **Matches Windows PE ransomware with common ransom strings, encryption indicators, or C2 URLs.**

⚡ **Limits file size to reduce false positives.**

```
import "pe"

rule Suspicious_Ransomware_Windows : ransomware windows
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects Windows ransomware using common strings and PE indicators"
        date = "2025-08-06"
        reference = "https://attack.mitre.org/techniques/T1486/"
        version = "1.0"

    strings:
        $s1 = "YOUR_FILES_ARE_ENCRYPTED" nocase
        $s2 = "RECOVER_FILES" nocase
        $s3 = "AES256" nocase
        $s4 = "RSA-2048" nocase
        $hex_pattern = { 68 ?? ?? ?? ?? 50 E8 ?? ?? ?? ?? 83 C4 04 }
        $url_c2 = "http://www.locky_fake_ransomware.org" nocase

    condition:
        uint16(0) == 0x5A4D and // PE header check
        any of ($s*) or // any ransom or crypto string
        $hex_pattern or
        $url_c2 and
        filesize < 10MB
}
```



Example #2

⚡ **Flags ransomware binaries that use packing, unusual section layouts, or entry point anomalies.**

```
import "pe"

rule Ransomware_PE_Anomalies : ransomware pe
{
  meta:
    author = "Technawi SOC Team"
    description = "Detects ransomware with suspicious PE characteristics"
    date = "2025-08-06"

  condition:
    pe.number_of_sections < 4 or          // unusually few sections
    pe.entry_point > pe.size_of_code or
    pe.section(".text").executable == false
}
```




Web Shell Detection

- ⚡ Web shells are malicious scripts (often .php, .asp, .aspx, .jsp) that allow attackers to execute commands on a compromised server.
- ⚡ They're widely used in post-exploitation for persistence, lateral movement, and data exfiltration.
- ⚡ Common indicators:
 - ⚡ Suspicious functions (eval, system, exec, shell_exec, cmd.exe)
 - ⚡ Obfuscation (base64_decode, str_rot13, XOR)
 - ⚡ Remote communication (curl, wget, socket)
 - ⚡ Short files with dense malicious logic



Example #1 – Generic Web Shell

```
rule Generic_WebShell_Detection
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects generic web shell code snippets"
        date = "2025-08-08"
        reference = "Common PHP/ASP web shell patterns"

    strings:
        // PHP shells
        $php_eval      = "eval(" nocase
        $php_exec       = "exec(" nocase
        $php_system    = "system(" nocase
        $php_shell      = "shell_exec" nocase
        $php_assert     = "assert(" nocase
        $php_base64     = "base64_decode(" nocase

        // ASP / ASPX shells
        $asp_process    = "System.Diagnostics.Process" nocase
        $asp_execute    = "Execute(" nocase
        $asp_shell      = "cmd.exe" nocase

        // Obfuscation / Backdoor markers
        $xor_pattern    = "XOR" nocase
        $rot13_pattern  = "str_rot13" nocase
        $pass_param     = "password=" nocase

    condition:
        (any of ($php_*)) or (any of ($asp_*)) or (2 of ($xor_pattern,$rot13_pattern,$pass_param))
}
```




Example #2 – Known PHP Web Shells

```
rule PHP_WebShell_Specific
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects known malicious PHP web shells like c99, r57, b374k"
        date = "2025-08-08"

    strings:
        $c99 = "c99shell" nocase
        $r57 = "r57shell" nocase
        $b374k = "b374k" nocase
        $webshell_title = "<title>php shell" nocase

    condition:
        any of them
}
```



Example #3 – ASP.NET Web Shells

```
rule ASPNET_WebShell
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects ASP.NET based web shells"
        date = "2025-08-08"

    strings:
        $asp1 = "System.Web.HttpContext.Current.Request" nocase
        $asp2 = "System.Diagnostics.Process.Start" nocase
        $asp3 = "cmd.exe /c" nocase
        $asp4 = "powershell.exe" nocase

    condition:
        any of ($asp1,$asp2,$asp3,$asp4)
}
```




Vulnerability Detection

- ⚡ YARA is typically used to detect malware, shells, and binaries, but it can also be adapted to detect common web vulnerabilities like SQL Injection (SQLi), Cross-Site Scripting (XSS), and Command Injection by searching for malicious payloads, exploit strings, or vulnerable code patterns.
- ⚡ YARA works on files, logs, and code repositories, so detection of vulnerabilities focuses on:
 - ⚡ Source code scanning → Detect vulnerable code (e.g., unsanitized SQL queries, eval() calls).
 - ⚡ Log analysis → Detect exploit attempts (e.g., UNION SELECT, <script> injection, ; cat /etc/passwd).
 - ⚡ Web payload hunting → Detect encoded/obfuscated payloads inside traffic captures, uploads, or WAF logs.



Example #1

⚡ Use on Apache/IIS logs, WAF alerts, or raw payload captures.

```
rule Detect_SQLi_Attempts
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects common SQL Injection attack patterns in logs or code"
        date = "2025-08-08"

    strings:
        $sqli_union    = "UNION SELECT" nocase
        $sqli_comment  = "-- " nocase
        $sqli_or        = "' OR '1'='1" nocase
        $sqli_drop      = "DROP TABLE" nocase
        $sqli_sleep     = "SLEEP(" nocase
        $sqli_hex       = "0x31303235343830303536" // encoded payload

    condition:
        any of them
}
```




Example #2

```
rule SQLi_Vulnerability_in_SourceCode
{
    meta:
        author = "Technawi SOC Team"
        description = "Detects potential SQL Injection vulnerabilities in source code by identifying unsafe SQL query patterns"
        date = "2025-08-08"
        reference = "OWASP SQL Injection Prevention"

    strings:
        // Common unsafe SQL query usage
        $php_mysql_query = "mysql_query(" nocase
        $php_mysqli_query = "mysqli_query(" nocase
        $php_pdo_query = "->query(" nocase

        // Concatenation patterns (insecure query building)
        $concat1 = "SELECT * FROM" nocase
        $concat2 = "INSERT INTO" nocase
        $concat3 = "UPDATE " nocase
        $concat4 = "DELETE FROM" nocase
        $concat_var = "\".$" // suspicious variable concatenation in query

        // ASP / .NET unsafe DB calls
        $asp_execute = "Execute(" nocase
        $asp_sql = "SELECT * FROM" nocase

    condition:
        // Must have SQL-related string + variable concatenation
        (any of ($php_mysql_query, $php_mysqli_query, $php_pdo_query, $asp_execute)) and
        (any of ($concat1, $concat2, $concat3, $concat4, $asp_sql)) and
        $concat_var
}
```



Arab Security Conference

2025
7th - 8th SEPTEMBER

Real World Scenarios



Arab Security Conference

2025
7th - 8th SEPTEMBER

Stealc Stealer



Demo #1 - Stealc

1. Both stealc samples size are less than 240KB

The screenshot shows a Windows File Explorer window titled 'stealc'. The address bar shows the path 'Start backup > stealc'. The left sidebar shows 'Home', 'Gallery', 'OneDrive', and 'Desktop'. The main area displays a table of files:

Name	Date modified	Type	Size
07b3c4a47ec2b0e62681dd4de6866b809a82262c45360b24a19e47b2b17ed5c9	9/6/2025 12:11 PM	Application	231 KB
716cf3d14949e2892a8a215c7d97ab4534a35af1ea09321fe8c8bae07ceb3dcf	9/6/2025 12:11 PM	Application	176 KB

At the bottom left, it says '2 items'. At the bottom right, there are icons for 'Details' and 'Compare'.



Demo #1 - Stealc

2. Using *PEStudio*, we found common strings in both samples as shown below

.rsrc	-	-	-	-	***
.rsrc	-	-	-	-	555
.rsrc	-	-	-	-	MMM
.rsrc	-	-	-	-	bbb
.text	-	-	-	-	Ah@
.rdata	-	-	-	-	(null)
.rdata	-	-	-	-	(((H
.rdata	-	-	-	-	h(((H
.rdata	-	-	-	-	yekel
.rdata	-	-	-	-	yir
.rdata	-	-	-	-	senewuparagoratiypevojura
.rsrc	-	-	-	-	TASUNIYUVOMOVEVOJUCUXO
.rsrc	-	-	-	-	/ P6pL
.rsrc	-	-	-	-	./KPip
.rsrc	-	-	-	-	/-P?pR
.rsrc	-	-	-	-	/ P6pL
.rsrc	-	-	-	-	./KPip
.rsrc	-	-	-	-	/-P?pR
.rsrc	-	-	-	-	/ P6pL
.rsrc	-	-	-	-	./KPip
.rsrc	-	-	-	-	/-P?pR



Demo #1 - Stealc

3. Using *Pebear*, we found the common bytes in both samples as shown below

Compare...

C:/Users/technawi/Desktop/stealc/716cf3d14949e2892a8a215c7d97ab4534a35af1ea09321fe8c8bae07ceb3dcf.exe

716cf3d14949e2892a8a215c7d97ab4534a35af1ea09321fe8c8bae07ceb3dcf.exe

- DOS Header
- DOS stub
- NT Headers
 - Signature
 - File Header
 - Optional Header
- Section Headers
- Sections
 - .text
 - EP = DB1
 - .rdata
 - .data
 - .rsrc

Relative (\$+) Hex View Next Diff

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	E8	07	22	00	00	E9	79	FF	FF	88	08	20	42	00	C3	
10	A1	A0	02	43	00	66	6A	14	52	85	C0	75	07	88	00	02
20	00	00	E8	06	8B	C6	7D	07	8B	C6	A3	A0	02	43	00	6A
30	04	50	E8	80	2F	00	00	59	59	A3	9C	F3	42	00	85	C0
40	75	15	6A	04	56	89	35	A0	02	43	00	59	67	2F	00	00
50	59	59	A3	9C	F3	42	00	85	C0	75	05	6A	1A	58	53	C3
60	83	D2	89	05	20	42	00	E9	05	A1	9C	F3	42	00	85	C0
70	02	83	C1	20	83	C2	04	81	F9	89	22	42	00	70	2A	CA
80	F3	52	83	D2	89	13	40	42	00	57	8B	C2	C1	F8	05	8B
90	04	89	80	F3	42	00	85	F3	83	27	13	C1	27	06	8B	04
A0	07	83	F8	F3	74	05	8B	C6	74	04	85	C0	75	02	83	31
B0	83	C1	20	42	81	F3	78	20	42	00	70	C5	5F	33	00	5E
C0	C3	E9	54	31	00	00	8D	89	47	42	00	00	74	05	E8	

C:/Users/technawi/Desktop/stealc/07b3c4a47ec2b0e62681dd4de6866b809a82262c45360b24a19e47b2b17ed5c9.exe

07b3c4a47ec2b0e62681dd4de6866b809a82262c45360b24a19e47b2b17ed5c9.exe

- DOS Header
- DOS stub
- NT Headers
 - Signature
 - File Header
 - Optional Header
- Section Headers
- Sections
 - .text
 - EP = 128F
 - .rdata
 - .data
 - .rsrc

Relative (\$+) Hex View Next Diff

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	E8	A3	92	00	00	E9	79	FF	FF	88	08	20	42	00	C3	
10	00	E8	7D	32	00	00	A3	78	52	43	00	83	C0	C3	C0	C0
20	C0	55	8B	8C	83	8C	08	83	E4	70	0D	1C	24	F3	05	75
30	04	24	E8	08	00	00	00	C9	C3	66	03	12	44	24	04	68
40	03	C5	C0	03	66	25	F3	75	66	2D	30	80	66	3D	C5	10
50	05	87	42	01	00	00	66	05	12	0D	50	4A	42	00	F2	05
60	59	C9	66	05	12	15	58	4A	42	00	F2	05	2D	D1	F2	05
70	58	CA	66	05	12	1D	70	4A	42	00	F2	05	5C	CA	66	05
80	28	15	60	4A	42	00	F2	05	59	D9	66	05	14	C9	81	C2
90	10	76	1C	00	F2	05	10	20	83	22	83	66	05	23	2D	40
A0	4A	42	00	8D	05	10	42	42	00	C1	22	05	03	C2	66	05



Demo #1 - Stealc

4. Now, we can write this simple YARA rule to detect this Malware.

```
rule stealc_detection {
  meta:
    author = "Technawi SOC Team"
    description = "Simple YARA rule to detect Stealc Stealer"
    date = "6/8/2025"
    hash1 = "07b3c4a47ec2b0e62681dd4de6866b809a82262c45360b24a19e47b2b17ed5c9"
    hash2 = "716cf3d14949e2892a8a215c7d97ab4534a35af1ea09321fe8c8bae07ceb3dcf"

  strings:
    $s1 = "senewuparagoratiyipevojura" wide
    $s2 = "TASUNIYUVOMOVEVOJUCUX0" wide
    $s3 = {E8 ?? ?? 00 00 E9 79 FE FF FF}

  condition:
    uint16(0) == 0x4d5a and filesize < 240KB and all of them
}
```



Arab Security Conference

2025
7th - 8th SEPTEMBER

AgentTesla



Demo #2 - AgentTesla

1. Extracting the strings inside the sample.

```
Windows PowerShell
PS C:\Users\technawi\Downloads\Strings> .\strings.exe 'C:\Users\technawi\Desktop\agent tesla\74921468edba01c46fcab0c265474fa65fc3c0f23363dd1223842b621db6fafc.exe' > 'C:\users\technawi\Desktop\agent tesla\strings.txt'
```



Demo #2 - AgentTesla

2. Extracting the strings inside the sample.

strings.txt

File Edit View

TargetFrameworkAttribute

System.Runtime.Versioning

ComVisibleAttribute

ParamArrayAttribute

CompilerGeneratedAttribute

ObsoleteAttribute

ExtensionAttribute

DebuggerStepThroughAttribute

AsyncStateMachineAttribute

ThreadStaticAttribute

DynamicAttribute

DebuggerHiddenAttribute

Command failed:

Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0

http://3.72.88.224/f1/Xluumkamo.mp4

CompressedBytes

AssemblyBytes

05heHseYcrQWgZ5Zxe.aEgH1E11QhbB7PdSyc

eQgR3vF40

loggerConfiguration

addScalar

addPolicy

scalarType

strings.txt

File Edit View

z\V

{Timestamp} [{Level}] {Message:1}{NewLine:1}{Exception:1}

hv{

{Timestamp:G} [{Level}] {Message:1}{NewLine:1}{Exception:1}

{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz} [{Level}] {Message:1}{NewLine:1}{Exception:1}

WrapNonExceptionThrows

GyazoGIF

Helpfeel Inc.

-Copyright

2023 Gyazo Team at Helpfeel Inc.

Gyazo

\$0a6a6d40-b9ee-4f07-9a4a-7a1fb217e771

5.5.3.0

.NETFramework,Version=v4.6

FrameworkDisplayName

.NET Framework 4.6

Use named arguments with this method to guarantee forwards-compatibility.

MPlease use WriteTo.Sink(new FileSink(path, new RawFormatter(), null)) instead

>Serilog.Verification.CustomizableVerifier+<EmitBatchAsync>d__3

_CorExMain

mscorlib.dll

Q;

R{-



Demo #2 - AgentTesla

3. YARA Rule based-on the strings.

```
rule AgentTesla_Detection : malware keylogger
{
    meta:
        author = "Technawi SOC Team"
        description = "A simple rule to detect AgentTesla"
        date = "08-08-2025"

    strings:
        $user-agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0"
        $c2_link = "http://3.72.88.224/f1/Xluumkamo.mp4"
        $dll = "mscoree.dll"
        $gyazo = "GyazoGIF"
        $helpfeel = "Helpfeel Inc."
        $dotnet_framework = ".NET Framework 4.6"

    condition:
        3 of them
}
```



Demo #2 - AgentTesla

4. By checking virus total, we can find the imphash for this sample.

74921468edba01c46fcab0c265474fa65fc3c0f23363dd1223842b621db6fafc

54 / 72

Community Score -70

54/72 security vendors flagged this file as malicious

74921468edba01c46fcab0c265474fa65fc3c0f23363dd1223842b621db6fafc

ref_1064_607301.exe

Size 165.12 KB Last Analysis Date 9 days ago

peexe checks-network-adapters overlay persistence detect-debug-environment assembly signed checks-user-input calls-wmi long-sleeps invalid-signature checks-bios spreader

Reanalyze Similar More

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 11

[Join our Community](#) and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

Basic properties ⓘ

MD5	634df01224f44b81e5f42bec836da17
SHA-1	1ab5d81824cdb83698b3f88898deb0519e887eea
SHA-256	74921468edba01c46fcab0c265474fa65fc3c0f23363dd1223842b621db6fafc
Vhash	21503655151240882a422a7
Authentihash	c582fd908a7a745517f7bf0cf404f1af659454f1071c52056db57a52366188da
Imphash	f34d5f2d4577ed6d9ceec516c1f5a744
SSDEEP	1536:GBfIDknO9lkfr65Gxz40RF1aIpILybbmGYFAMJIGvdL3Q5+g0yppuPsmWwp0LRNE:KHy4gO4IpY7AmRLgkgJY0knOhpC/j
TLSH	T10BF37032779781E1CF845B77C8A6053C17A8ED83A653D64A29CAFFD93833F668D40216



Demo #2 - AgentTesla

- ⚡ **imphash (Import Hash) is a hash value calculated from the Import Address Table (IAT) of a Portable Executable (PE) file (i.e., Windows .exe or .dll).**
- ⚡ **Specifically, it's generated by:**
 - ⚡ **Listing all imported DLLs and functions (in order).**
 - ⚡ **Concatenating them into a string.**
 - ⚡ **Applying MD5 hashing to that string.**
- 👉 **This produces a unique(ish) fingerprint of how a binary imports external functions.**



Demo #2 - AgentTesla

5. More advanced rule based-on both static strings and PE file info.

```
rule AgentTesla_Detection_Advanced : malware keylogger
{
    meta:
        author = "Technawi SOC Team"
        description = "A simple rule to detect AgentTesla"
        date = "08-08-2025"

    strings:
        $user-agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0"
        $c2_link = "http://3.72.88.224/f1/Xluumkamo.mp4"
        $dll = "mscoree.dll"
        $gyazo = "GyazoGIF"
        $helpfeel = "Helpfeel Inc."
        $dotnet_framework = ".NET Framework 4.6"

    condition:
        unit16(0) == 0x5A4D and
        pe.imphash() == "f34d5f2d4577ed6d9ceec516c1f5a744" and
        filesize < 200000 and
        3 of ($user-agent, $c2_link, $dll, $gyazo, $helpfeel, $dotnet_framework)
}
```




Arab Security Conference

9th

Round

THANK YOU